

LAB REPORT

Generation of Variable Frequency Triangular Wave using DAC and UART (AT89S52)

1. Objective

To generate a triangular waveform of amplitude ± 2.5 V using an 8-bit DAC with variable frequency (100 Hz – 500 Hz), controlled via UART communication at 19200 baud.

2. Components Required

AT89S52 Microcontroller

8-bit DAC0808

Serial Interface (USB-UART)

PC with Keil IDE

3. Theory

DAC Input Port Logic

Triangular wave is generated by incrementing and decrementing DAC values.

For 8-bit DAC:

Range = 0–255

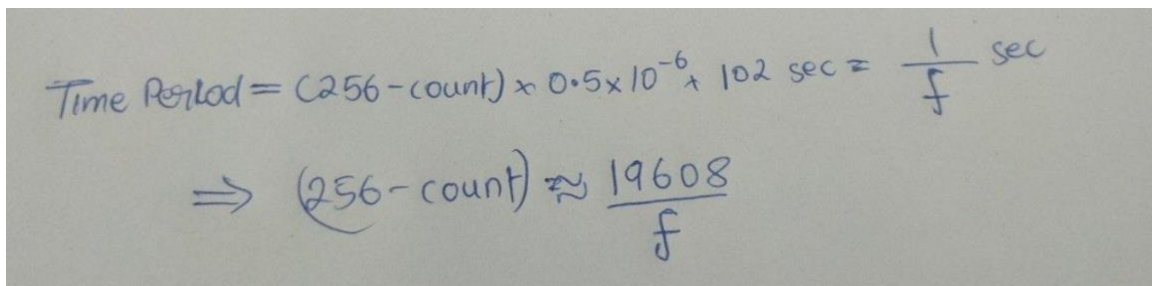
Voltage range = -2.5V to +2.5V

Resolution ≈ 19.53 mV

Here it is asked to generate a 51-point triangular wave. So the procedure followed was to increment the DAC input by 5 for positive-sloped ramp and decrement it by 5 for negative-sloped ramp since $255/5 = 51!$

Incorporating variable frequency into the program

To accommodate different frequencies, we modify the timer overflow rate according to the given frequency. The working formula is as follows:


$$\text{Time Period} = (256 - \text{count}) \times 0.5 \times 10^{-6} \times 102 \text{ sec} = \frac{1}{f} \text{ sec}$$
$$\Rightarrow (256 - \text{count}) \approx \frac{19608}{f}$$

4. Algorithm

1. Initialize UART (19200 baud).
2. Receive frequency through UART.
3. Calculate the corresponding count from the given frequency.
4. Generate triangular waveform (0→255→0) through timer0 ISR routine.
5. Repeat continuously with help of only ISR routines.

5. Program

```
cycle equ 30h
```

```
org 0000h  
ljmp Main
```

```
org 0000Bh  
ljmp T0_ISR
```

```
org 023h  
jmp serial_port_isr
```

```
org 0040h  
Main:  
DIV_H    DATA 20H ; Dividend high byte  
DIV_L    DATA 21H ; Dividend low byte  
DVSR_H   DATA 02h ; Divisor high byte  
DVSR_L   DATA 03h ; Divisor low byte  
QUOT_H   DATA 24H ; Quotient high byte  
QUOT_L   DATA 25H ; Quotient low byte  
REM_H    DATA 26H ; Remainder high byte  
REM_L    DATA 27H ; Remainder low byte
```

```
mov p1,#00h  
mov r0,#00h  
mov cycle,#00h  
mov 20h,#4ch    //4c98h is 19608d  
mov 21h,#98h
```

```
mov pcon,#80h  
mov tmod,#22h  
mov th1,#(256-7)  
mov tl1,th1
```

```
mov scon,#50h
```

```
mov th0,#1  
mov tl0,th0
```

```
setb tr1  
setb tr0
```

```
setb ea  
setb et0
```

```
setb es
```

```
setb ps
```

```
waiting: jmp waiting
```

```
Serial_Jump:
```

```
jnb ri,Serial_Jump
```

```
mov b,sbuf
```

```
mov r2,b
```

```
clr ri
```

```
Serial_Jump2:
```

```
jnb ri,Serial_Jump2
```

```
mov b,sbuf
```

```
mov r3,b
```

```
clr ri
```

```
acall Divider_Routine
```

```
mov a,25h
```

```
cpl a
```

```
mov 25h,a
```

```
inc 25h
```

```
mov th0,25h
```

```
mov tl0,th0
```

```
sjmp Serial_Jump
```

```
org 0100h
```

```
T0_ISR:
```

```
push acc
```

```
push psw
```

```
mov a, r0
```

```
mov p1,a
```

```
jb cycle, Decrement
```

```

    cjne r0,#255,Exit
    setb cycle
    mov r0,#250
    sjmp Final_Exit
Exit:
    add a,#5
    mov r0,a
    sjmp Final_Exit

Decrement:
    cjne r0,#00h,Dec_Exit
    clr cycle
    mov r0,#05
    sjmp Final_Exit
Dec_Exit:
    clr c
    subb a,#5
    mov r0,a
Final_Exit:
    pop psw
    pop acc
    reti

```

```

org 0200h
serial_port_isr:
    push acc
    mov b,sbuf
    mov r2,b
    clr ri
Serial_Jump_2:
    jnb ri,Serial_Jump_2
    mov b,sbuf
    mov r3,b
    clr ri
    acall Divider_Routine
    mov a,25h
    cpl a
    mov 25h,a
    inc 25h
    mov th0,25h
    mov tl0,th0
    pop acc

```

reti

org 0300h

Divider_Routine:

```
MOV QUOT_H, #0
MOV QUOT_L, #0
MOV REM_H, #0
MOV REM_L, #0
MOV DIV_H, #4cH
MOV DIV_L, #98H
clr c
```

```
MOV R7, #16 ; 16 bits to process
```

DIV_LOOP:

```
; Shift dividend left (MSB goes into C)
```

```
MOV A, DIV_L
RLC A
MOV DIV_L, A
MOV A, DIV_H
RLC A
MOV DIV_H, A
```

```
; Shift remainder left and bring in C
```

```
MOV A, REM_L
RLC A
MOV REM_L, A
MOV A, REM_H
RLC A
MOV REM_H, A
```

```
; Compare remainder with divisor
```

```
clr C
MOV A, REM_H
SUBB A, DVSR_H
```

```
JC NO_SUBTRACT ; If remainder < divisor, skip
JNZ subtract
```

```

MOV A, REM_L
CLR C
SUBB A, DVSR_L
JC NO_SUBTRACT
subtract: MOV A, REM_L ; Subtract divisor from remainder
          CLR C
          SUBB A, DVSR_L
          MOV REM_L, A
          MOV A, REM_H
          SUBB A, DVSR_H
          MOV REM_H, A

          ; Set quotient bit
SETB C
          MOV A, QUOT_L
          RLC A
          MOV QUOT_L, A
          MOV A, QUOT_H
          RLC A
          MOV QUOT_H, A

          SJMP NEXT_BIT

NO_SUBTRACT:
          clr c
          ; Shift quotient left by 1 without setting bit
          MOV A, QUOT_L
          RLC A
          MOV QUOT_L, A
          MOV A, QUOT_H
          RLC A
          MOV QUOT_H, A

NEXT_BIT:
          DJNZ R7, DIV_LOOP
          RET

end

```

6. Circuit Diagram

This can be only updated after the verification is done in the labrotary

7. Simulation/Result

Expected to observe the waveform through an oscilloscope during verification hence will be updated later. If needed, the keil simulation will be attached if required for an alternative

8. Observations

The code is basically dividing a 15 bit number with a 9 bit number in it's division routine. Henceforth, it cannot incorporate the **corresponding remainder** into it's subsequent calculation.

Therefore, the frequency which gives a higher remainder, will result in a bigger error.

Example: 469HZ

9. Result

Triangular waveform of $\pm 2.5V$ successfully generated with variable frequency(100Hz-500Hz) control via UART.

10. Conclusion

The experiment demonstrates successful waveform generation using AT89S52 and UART-based control.